

Laboratori LI. Tardor 2025

- **Professor:** Antoni Lozano
- **C/E:** antoni.lozano @ upc.edu
- **Despatx:** 233, edifici Ω

Informació general del curs

- **Pàgina:** <https://www.cs.upc.edu/~li/>
Pràctiques de laboratori, apunts, exàmens (resolts)
- **Racó:** avisos i lliurament de pràctiques

Pràctiques

- Hi ha 6 **pràctiques** i hi dediquem 2 dies per pràctica:
 - **P1:** SAT solver
 - **P2:** Introducció a Prolog
 - **P3:** Codificació en SAT
 - **P4:** Optimització en SAT
 - **P5:** Prolog avançat
 - **P6:** Constrained Logic Programming
- Cal entendre-les i també fer-les!

Parcials de laboratori

Hi ha 2 parcials de laboratori:

1 Parcial 1: 04/11/25

2 Parcial 2: 22/12/25

Pràctica 1: SAT solver

Lògica en la Informàtica

FIB

Antoni Lozano

(incorpora exemple de R. Nieuwenhuis)

Q1 2025–2026

Objectiu

Aquesta primera pràctica té com a objectius:

- enfrontar-se a un problema NP-complet com SAT
- adquirir uns coneixements bàsics sobre els SAT solvers

Per fer-ho, caldrà millorar l'eficiència d'un SAT solver que es proporciona.

Conceptes bàsics

- **SAT**: decidir si una fórmula de lògica proposicional té un model
- Lògica proposicional
 - Sintaxi. Les fórmules es construeixen amb símbols de predicat

$$P = \{p, q, r, \dots\}$$

i les connectives

$\wedge$ (conjunció) , $\vee$ (disjunció) , $\neg$ (negació)

- Semàntica. Una interpretació és una funció $I : P \longrightarrow \{0, 1\}$.
Diem que I satisfà una fórmula F ($I \models F$) si $\text{eval}_I(F) = 1$.

Conceptes bàsics

- **SAT**: decidir si una fórmula de lògica proposicional té un model
- **Lògica proposicional**
 - **Sintaxi**. Les **fórmules** es construeixen amb símbols de predicat

$$P = \{p, q, r, \dots\}$$

i les connectives

$\wedge$ (conjunció) , $\vee$ (disjunció) , $\neg$ (negació)

- **Semàntica**. Una **interpretació** és una funció $I : P \longrightarrow \{0, 1\}$.
Diem que I satisfà una fórmula F ($I \models F$) si $\text{eval}_I(F) = 1$.

Conceptes bàsics

- Exemple de fórmula F : $p \wedge ((q \vee \neg r) \wedge ((\neg p \vee r) \wedge \neg q))$
- Exemple d'interpretació: $I(p) = 0, I(q) = 1, I(r) = 0$
- $\text{eval}_I(F) = 0 \wedge ((1 \vee \neg 0) \wedge ((\neg 0 \vee 0) \wedge \neg 1))$
 $= 0 \wedge ((1 \vee 1) \wedge ((1 \vee 0) \wedge 0))$
 $= 0 \wedge (1 \wedge (1 \wedge 0))$
 $= 0 \wedge (1 \wedge 0)$
 $= 0 \wedge 0$
 $= 0$
- Per tant, F és falsa amb la interpretació I

Conceptes bàsics

- Exemple de fórmula F : $p \wedge ((q \vee \neg r) \wedge ((\neg p \vee r) \wedge \neg q))$
- Exemple d'interpretació: $I(p) = 0, I(q) = 1, I(r) = 0$
- $eval_I(F) = 0 \wedge ((1 \vee \neg 0) \wedge ((\neg 0 \vee 0) \wedge \neg 1))$
 $= 0 \wedge ((1 \vee 1) \wedge ((1 \vee 0) \wedge 0))$
 $= 0 \wedge (1 \wedge (1 \wedge 0))$
 $= 0 \wedge (1 \wedge 0)$
 $= 0 \wedge 0$
 $= 0$
- Per tant, F és falsa amb la interpretació I

Conceptes bàsics

- Exemple de fórmula F : $p \wedge ((q \vee \neg r) \wedge ((\neg p \vee r) \wedge \neg q))$
- Exemple d'interpretació: $I(p) = 0, I(q) = 1, I(r) = 0$
- $\text{eval}_I(F) = 0 \wedge ((1 \vee \neg 0) \wedge ((\neg 0 \vee 0) \wedge \neg 1))$
 $= 0 \wedge ((1 \vee 1) \wedge ((1 \vee 0) \wedge 0))$
 $= 0 \wedge (1 \wedge (1 \wedge 0))$
 $= 0 \wedge (1 \wedge 0)$
 $= 0 \wedge 0$
 $= 0$
- Per tant, F és falsa amb la interpretació I

Conceptes bàsics

- Exemple de fórmula F : $p \wedge ((q \vee \neg r) \wedge ((\neg p \vee r) \wedge \neg q))$
- Exemple d'interpretació: $I(p) = 0, I(q) = 1, I(r) = 0$
- $\text{eval}_I(F) = 0 \wedge ((1 \vee \neg 0) \wedge ((\neg 0 \vee 0) \wedge \neg 1))$
 $= 0 \wedge ((1 \vee 1) \wedge ((1 \vee 0) \wedge 0))$
 $= 0 \wedge (1 \wedge (1 \wedge 0))$
 $= 0 \wedge (1 \wedge 0)$
 $= 0 \wedge 0$
 $= 0$
- Per tant, F és falsa amb la interpretació I

Conceptes bàsics

- Exemple de fórmula F : $p \wedge ((q \vee \neg r) \wedge ((\neg p \vee r) \wedge \neg q))$
- Exemple d'interpretació: $I(p) = 0, I(q) = 1, I(r) = 0$
- $\text{eval}_I(F) = 0 \wedge ((1 \vee \neg 0) \wedge ((\neg 0 \vee 0) \wedge \neg 1))$
 $= 0 \wedge ((1 \vee 1) \wedge ((1 \vee 0) \wedge 0))$
 $= 0 \wedge (1 \wedge (1 \wedge 0))$
 $= 0 \wedge (1 \wedge 0)$
 $= 0 \wedge 0$
 $= 0$
- Per tant, F és falsa amb la interpretació I

Conceptes bàsics

- Exemple de fórmula F : $p \wedge ((q \vee \neg r) \wedge ((\neg p \vee r) \wedge \neg q))$
- Exemple d'interpretació: $I(p) = 0, I(q) = 1, I(r) = 0$
- $\text{eval}_I(F) = 0 \wedge ((1 \vee \neg 0) \wedge ((\neg 0 \vee 0) \wedge \neg 1))$
 $= 0 \wedge ((1 \vee 1) \wedge ((1 \vee 0) \wedge 0))$
 $= 0 \wedge (1 \wedge (1 \wedge 0))$
 $= 0 \wedge (1 \wedge 0)$
 $= 0 \wedge 0$
 $= 0$
- Per tant, F és falsa amb la interpretació I

Conceptes bàsics

- Exemple de fórmula F : $p \wedge ((q \vee \neg r) \wedge ((\neg p \vee r) \wedge \neg q))$
- Exemple d'interpretació: $I(p) = 0, I(q) = 1, I(r) = 0$
- $\text{eval}_I(F) = 0 \wedge ((1 \vee \neg 0) \wedge ((\neg 0 \vee 0) \wedge \neg 1))$
 $= 0 \wedge ((1 \vee 1) \wedge ((1 \vee 0) \wedge 0))$
 $= 0 \wedge (1 \wedge (1 \wedge 0))$
 $= 0 \wedge (1 \wedge 0)$
 $= 0 \wedge 0$
 $= 0$
- Per tant, F és falsa amb la interpretació I

Conceptes bàsics

- Exemple de fórmula F : $p \wedge ((q \vee \neg r) \wedge ((\neg p \vee r) \wedge \neg q))$
- Exemple d'interpretació: $I(p) = 0, I(q) = 1, I(r) = 0$
- $\text{eval}_I(F) = 0 \wedge ((1 \vee \neg 0) \wedge ((\neg 0 \vee 0) \wedge \neg 1))$
 $= 0 \wedge ((1 \vee 1) \wedge ((1 \vee 0) \wedge 0))$
 $= 0 \wedge (1 \wedge (1 \wedge 0))$
 $= 0 \wedge (1 \wedge 0)$
 $= 0 \wedge 0$
 $= 0$
- Per tant, F és falsa amb la interpretació I

Conceptes bàsics

- Exemple de fórmula F : $p \wedge ((q \vee \neg r) \wedge ((\neg p \vee r) \wedge \neg q))$
- Exemple d'interpretació: $I(p) = 0, I(q) = 1, I(r) = 0$
- $\text{eval}_I(F) = 0 \wedge ((1 \vee \neg 0) \wedge ((\neg 0 \vee 0) \wedge \neg 1))$
 $= 0 \wedge ((1 \vee 1) \wedge ((1 \vee 0) \wedge 0))$
 $= 0 \wedge (1 \wedge (1 \wedge 0))$
 $= 0 \wedge (1 \wedge 0)$
 $= 0 \wedge 0$
 $= 0$
- Per tant, F és falsa amb la interpretació I

Conceptes bàsics

- Totes les interpretacions de $F = p \wedge ((q \vee \neg r) \wedge ((\neg p \vee r) \wedge \neg q))$:

p	q	r	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Conceptes bàsics

- Si hi ha n símbols, quantes interpretacions hi ha?
- Una interpretació que satisfà una fórmula se'n diu **model**.
- F és **insatisfactible**, no té cap model.
- Quants models tenen les fórmules següents?

$$G = x_1 \wedge x_2 \wedge \cdots \wedge x_n$$

$$H = (\neg x_1 \vee x_2) \wedge (x_1 \wedge \neg x_2) \wedge x_3 \wedge \cdots \wedge x_n$$

Conceptes bàsics

- Si hi ha n símbols, quantes interpretacions hi ha?
- Una interpretació que satisfà una fórmula se'n diu **model**.
- F és **insatisfactible**, no té cap model.
- Quants models tenen les fórmules següents?

$$G = x_1 \wedge x_2 \wedge \cdots \wedge x_n$$

$$H = (\neg x_1 \vee x_2) \wedge (x_1 \wedge \neg x_2) \wedge x_3 \wedge \cdots \wedge x_n$$

Conceptes bàsics

- Si hi ha n símbols, quantes interpretacions hi ha?
- Una interpretació que satisfà una fórmula se'n diu **model**.
- F és **insatisfactible**, no té cap model.
- Quants models tenen les fórmules següents?

$$G = x_1 \wedge x_2 \wedge \cdots \wedge x_n$$

$$H = (\neg x_1 \vee x_2) \wedge (x_1 \wedge \neg x_2) \wedge x_3 \wedge \cdots \wedge x_n$$

Conceptes bàsics

- Si hi ha n símbols, quantes interpretacions hi ha?
- Una interpretació que satisfà una fórmula se'n diu **model**.
- F és **insatisfactible**, no té cap model.
- Quants models tenen les fórmules següents?

$$G = x_1 \wedge x_2 \wedge \cdots \wedge x_n$$

$$H = (\neg x_1 \vee x_2) \wedge (x_1 \wedge \neg x_2) \wedge x_3 \wedge \cdots \wedge x_n$$

Conceptes bàsics

- Si hi ha n símbols, quantes interpretacions hi ha?
- Una interpretació que satisfà una fórmula se'n diu **model**.
- F és **insatisfactible**, no té cap model.
- Quants models tenen les fórmules següents?

$$G = x_1 \wedge x_2 \wedge \cdots \wedge x_n \quad (1)$$

$$H = (\neg x_1 \vee x_2) \wedge (x_1 \wedge \neg x_2) \wedge x_3 \wedge \cdots \wedge x_n \quad (2)$$

SAT solvers

Els SAT solvers consideren les fórmules d'entrada en **forma normal conjuntiva** (CNF):

- Un **literal** és un símbol de predicat afirmat o negat
- Una **clàusula** és una disjunció de literals
- Una **fórmula booleana** en CNF és una conjunció de clàusules

Exemple

- Símbols: p, q, r
- Literals: $p, \neg p, q, \neg q, r, \neg r$
- Clàusules: $(p), (q \vee \neg r), (\neg p \vee r), (\neg q)$
- Fórmula en CNF: $F = (p) \wedge (q \vee \neg r) \wedge (\neg p \vee r) \wedge (\neg q)$

SAT solvers

Els SAT solvers consideren les fórmules d'entrada en **forma normal conjuntiva** (CNF):

- Un **literal** és un símbol de predicat afirmat o negat
- Una **clàusula** és una disjunció de literals
- Una **fórmula booleana** en CNF és una conjunció de clàusules

Exemple

- Símbols: p, q, r
- Literals: $p, \neg p, q, \neg q, r, \neg r$
- Clàusules: $(p), (q \vee \neg r), (\neg p \vee r), (\neg q)$
- Fórmula en CNF: $F = (p) \wedge (q \vee \neg r) \wedge (\neg p \vee r) \wedge (\neg q)$

SAT solvers

El format d'entrada (DIMACS) per al nostre SAT solver és:

```

p cnf 300 1275
-255 15 257 0
239 1 115 0
100 -118 -152 0
-32 93 20 0
272 41 -269 0
-220 -251 -53 0
-9 254 129 0
-38 -71 -127 0
-52 113 186 0
-175 -281 -291 0
  
```

literal $\neg x_{220}$

variables

clàusules

final de clàusula

SAT solvers

Algorismes per decidir la satisfactibilitat:

- **Cerca exhaustiva**: enumerar totes les interpretacions i, per a cadascuna, avaluar la fórmula. Avaluar la fórmula donada la interpretació és lineal, però tenim 2^n interpretacions.
- **DPLL-based SAT Solvers**
(DPLL: Davis–Putnam–Logemann–Loveland backtracking algorithm)
- **CDCL-based SAT Solvers** fets per Intel, Microsoft. . .
(CDCL: Conflict-Driven Clause-Learning backtracking/backjumping algorithm)

Exemple

Veurem la presentació de **backtrackingForSAT.pdf**.

SAT solvers

Algorismes per decidir la satisfactibilitat:

- **Cerca exhaustiva**: enumerar totes les interpretacions i, per a cadascuna, avaluar la fórmula. Avaluar la fórmula donada la interpretació és lineal, però tenim 2^n interpretacions.
- **DPLL-based SAT Solvers**
(DPLL: Davis–Putnam–Logemann–Loveland backtracking algorithm)
- **CDCL-based SAT Solvers** fets per Intel, Microsoft. . .
(CDCL: Conflict-Driven Clause-Learning backtracking/backjumping algorithm)

Exemple

Veurem la presentació de **backtrackingForSAT.pdf**.

A toy backtracking algorithm for SAT: an example

Here we denote variables by numbers and negation by overlining.

The model stack (the partial interpretation being explored) grows to the right.

Five clauses:

A toy backtracking algorithm for SAT: an example

Here we denote variables by numbers and negation by overlining.

The model stack (the partial interpretation being explored) grows to the right.

Five clauses:

$$\overline{1} \vee 2, \overline{3} \vee 4, \overline{4} \vee \overline{2} \vee \overline{1} \vee \overline{7}, \overline{5} \vee \overline{6}, 6 \vee \overline{5} \vee \overline{2}$$

A toy backtracking algorithm for SAT: an example

Here we denote variables by numbers and negation by overlining.

The model stack (the partial interpretation being explored) grows to the right.

Model Stack: **Five clauses:**

$\overline{1} \vee 2, \overline{3} \vee 4, \overline{4} \vee \overline{2} \vee \overline{1} \vee \overline{7}, \overline{5} \vee \overline{6}, 6 \vee \overline{5} \vee \overline{2} \Rightarrow (\text{Decide})$

1

$\overline{1} \vee 2, \overline{3} \vee 4, \overline{4} \vee \overline{2} \vee \overline{1} \vee \overline{7}, \overline{5} \vee \overline{6}, 6 \vee \overline{5} \vee \overline{2} \Rightarrow$

A toy backtracking algorithm for SAT: an example

Here we denote variables by numbers and negation by overlining.

The model stack (the partial interpretation being explored) grows to the right.

Model Stack: **Five clauses:**

	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	

A toy backtracking algorithm for SAT: an example

Here we denote variables by numbers and negation by overlining.

The model stack (the partial interpretation being explored) grows to the right.

Model Stack: **Five clauses:**

	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	

A toy backtracking algorithm for SAT: an example

Here we denote variables by numbers and negation by overlining.

The model stack (the partial interpretation being explored) grows to the right.

Model Stack: **Five clauses:**

	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	

A toy backtracking algorithm for SAT: an example

Here we denote variables by numbers and negation by overlining.

The model stack (the partial interpretation being explored) grows to the right.

Model Stack: **Five clauses:**

	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4 $\bar{7}$	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	

A toy backtracking algorithm for SAT: an example

Here we denote variables by numbers and negation by overlining.

The model stack (the partial interpretation being explored) grows to the right.

Model Stack: **Five clauses:**

	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4 $\bar{7}$	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3 4 $\bar{7}$ 5	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	

A toy backtracking algorithm for SAT: an example

Here we denote variables by numbers and negation by overlining.

The model stack (the partial interpretation being explored) grows to the right.

Model Stack: **Five clauses:**

	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4 $\bar{7}$	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3 4 $\bar{7}$ 5	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4 $\bar{7}$ 5 $\bar{6}$	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$		CONFLICT!

A toy backtracking algorithm for SAT: an example

Here we denote variables by numbers and negation by overlining.

The model stack (the partial interpretation being explored) grows to the right.

Model Stack: **Five clauses:**

	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4 $\bar{7}$	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3 4 $\bar{7}$ 5	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4 $\bar{7}$ 5 $\bar{6}$	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Backtrack)

A toy backtracking algorithm for SAT: an example

Here we denote variables by numbers and negation by overlining.

The model stack (the partial interpretation being explored) grows to the right.

Model Stack: **Five clauses:**

	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4 $\bar{7}$	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3 4 $\bar{7}$ 5	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4 $\bar{7}$ 5 $\bar{6}$	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Backtrack)
1 2 3 4 $\bar{7}$ 5	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$		

A toy backtracking algorithm for SAT: an example

Here we denote variables by numbers and negation by overlining.

The model stack (the partial interpretation being explored) grows to the right.

Model Stack: **Five clauses:**

	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4 $\bar{7}$	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3 4 $\bar{7}$ 5	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4 $\bar{7}$ 5 $\bar{6}$	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Backtrack)
1 2 3 4 $\bar{7}$ 5	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)

A toy backtracking algorithm for SAT: an example

Here we denote variables by numbers and negation by overlining.

The model stack (the partial interpretation being explored) grows to the right.

Model Stack: **Five clauses:**

	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4 $\bar{7}$	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3 4 $\bar{7}$ 5	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(UnitPropagate)
1 2 3 4 $\bar{7}$ 5 $\bar{6}$	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Backtrack)
1 2 3 4 $\bar{7}$ 5	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$	$\Rightarrow$	(Decide)
1 2 3 4 $\bar{7}$ 5 6	$\bar{1} \vee 2, \bar{3} \vee 4, \bar{4} \vee \bar{2} \vee \bar{1} \vee \bar{7}, \bar{5} \vee \bar{6}, 6 \vee \bar{5} \vee \bar{2}$		solution found!

SAT solvers

En aquest curs:

- 1 Triarem un problema P qualsevol
- 2 El codificarem com una fórmula proposicional F_P
- 3 Cridarem a un SAT solver amb F_P com a entrada
- 4 Convertirem el model que ens doni el SAT solver en la solució de P

Main de SAT-alumnes.cpp

```

int main(){
    readClauses(); // reads numVars, numClauses and clauses
    model.resize(numVars+1,UNDEF);
    indexOfNextLitToPropagate = 0;
    decisionLevel = 0;

    // Take care of initial unit clauses, if any
    for (uint i = 0; i < numClauses; ++i)
        if (clauses[i].size() == 1){
            int lit = clauses[i][0];
            int val = currentValueInModel(lit);
            if (val == FALSE) {cout << "UNSATISFIABLE" << endl; return 10;}
            else if (val == UNDEF) setLiteralToTrue(lit);
        }

    // DPLL algorithm
    while (true){
        while (! propagateGivesConflict()) {
            if (decisionLevel == 0){ cout << "UNSATISFIABLE" << endl; return 10;}
            backtrack();
        }
        int decisionLit = getNextDecisionLiteral();
        if (decisionLit == 0){ checkmodel(); cout << "SATISFIABLE" << endl; return 20; }
        // start new decision level:
        modelStack.push_back(0); // push mark indicating new DL
        ++indexOfNextLitToPropagate;
        ++decisionLevel;
        setLiteralToTrue(decisionLit); // now push decisionLit on top of the mark
    }
}

```

fer unit propagate fins
que ja no sigui possible



Com treballar la pràctica

- 1 Estudiant el funcionament del SAT solver que us proporcionem `SAT-alumnes.cpp`.
- 2 Seguint pas a pas les instruccions de la pàgina de LI:
<https://www.cs.upc.edu/~li/practical.html>
 - Si treballeu en portàtil, instal·leu **kissat** des de
<http://fmv.jku.at/kissat/>
- 3 En concret, caldrà
 - (a) millorar `propagateGivesConflict()` (la **unit propagation**) fent servir **OccurLists**, llistes on s'indica en quines clàusules hi apareix negat cada literal.
 - (b) millorar `getNextDecisionLiteral()` perquè decideixi primer sobre variables que poden tenir més impacte. Per exemple, aquelles que
 - **apareguin més sovint** en les clàusules inicials
 - provoquin més **conflictes**

Milllores

(a): millorar la propagació

En lloc de recórrer totes les clàusules, fer servir **OccurLists** amb les clàusules rellevants per la propagació actual.

```
bool propagateGivesConflict ( ) {
    while ( indexOfNextLitToPropagate < modelStack.size() ) {
        ++indexOfNextLitToPropagate;
        for (uint i = 0; i < numClauses; ++i) {
            bool someLitTrue = false;
            int numUndefs = 0;
            int lastLitUndef = 0;
            for (uint k = 0; not someLitTrue and k < clauses[i].size(); ++k){
                int val = currentValueInModel(clauses[i][k]);
                if (val == TRUE) someLitTrue = true;
                else if (val == UNDEF){ ++numUndefs; lastLitUndef = clauses[i][k]; }
            }
            if (not someLitTrue and numUndefs == 0) return true; // conflict! all lits false
            else if (not someLitTrue and numUndefs == 1) setLiteralToTrue(lastLitUndef);
        }
    }
    return false;
}
```

Milllores

(b): millorar la decisió del literal

En lloc de triar la primera variable indefinida, triar una variable amb “impacte”.

```
// Heuristic for finding the next decision literal:
int getNextDecisionLiteral(){
    for (uint i = 1; i <= numVars; ++i) // stupid heuristic:
        if (model[i] == UNDEF) return i; // returns first UNDEF var, positively
    return 0; // reurns 0 when all literals are defined
}
```

Milllores

Algunes **heurístiques** per `getNextDecisionLiteral()`:

- 1 Per cada variable, tenir un **comptador d'aparicions** en les clàusules inicials. Triar la variable indefinida amb el comptador més alt.
- 2 Tenir un comptador per variable x que compti **quantes vegades ha aparegut x en un conflicte**:
 - Cada cop que es detecta un conflicte en una clàusula, s'incrementen els comptadors de les variables de la clàusula.
 - Donar més pes als **conflictes recents**. Per exemple, dividir tots els comptadors per 2 cada N conflictes. (Proveu valors alts de N com 50.000, 100.000,...)

Dificultat dels problemes

Per què els problemes de 300 variables de `random3SAT` són tan difícils?
(quan resolem sudokus amb 720 var. instantàniament...)

- 1 Perquè les clàusules estan **generades a l'atzar**
- 2 Pel **nombre de clàusules** generades.
 - Si es generen poques clàusules, la fórmula és probablement satisfactible
 - Si es generen moltes clàusules, la fórmula és probablement insatisfactible
 - Quantes clàusules cal generar perquè la probabilitat que una fórmula 3SAT de n variables sigui satisfactible sigui $1/2$?

$\approx 4.25n$ (trobadura experimentalment)

Dificultat dels problemes

Per què els problemes de 300 variables de `random3SAT` són tan difícils?
(quan resollem sudokus amb 720 var. instantàniament...)

- ❶ Perquè les clàusules estan **generades a l'atzar**
- ❷ Pel **nombre de clàusules** generades.
 - Si es generen poques clàusules, la fórmula és probablement satisfactible
 - Si es generen moltes clàusules, la fórmula és probablement insatisfactible
 - Quantes clàusules cal generar perquè la probabilitat que una fórmula 3SAT de n variables sigui satisfactible sigui $1/2$?
 $\approx 4.25n$ (trobadura experimentalment)

Enunciat de la pràctica

Pràctica 1: SAT solver

Modifica el SAT solver [SAT-alumnes.cpp](#) per a tractar de la manera més eficient que puguis els exemples de [random3SAT.tar.gz](#). Teniu una presentació descrivint l'algorisme [aquí](#), amb [l'exemple](#) representat en format *DIMACS* pel SAT solver.

Per a aconseguir això, millora l'heurística de decisió i la velocitat de propagació:

1. Propagació.

A cada volta del bucle de `propagateGivesConflict`, cal propagar que `modelStack[indexOfNextLiteralToPropagate]` s'ha tornat cert.

Per fer-ho, podeu fer servir *occur lists*: per cada literal `l`, tenim una llista `occurList[l]` que conté els índexs de les clàusules on apareix `l`.

Quan cal propagar que un literal `l` és cert, el que fem és visitar les clàusules de `occurList[-l]` i mirem si n'hi ha cap que s'hagi tornat falsa (conflicte!) o unitària (propagació!).

Enunciat de la pràctica

2. Heurística de decisió.

a. Tenim, per cada variable, un comptador del nombre de clàusules on apareix.
En `getNextDecisionLiteral`, quan hem de decidir, escollim la variable *indefinida* amb aquest comptador més alt.

b. Tenim, per cada variable, un comptador del nombre de conflictes en què ha aparegut. Quan hem de decidir, escollim la variable indefinida amb aquest comptador més alt.
Una millora que es pot fer és donar més pes als conflictes recents. Per exemple, cada X conflictes, dividim els comptadors per la meitat.

Adjuntem un **script** que compara l'eficiència del teu solver amb un altre, **kissat**, executant tots dos en la mateixa màquina. Sobre els problemes insatisfactibles de 300 variables, **kissat** mai hauria de ser més de 20 vegades més ràpid que el teu solver, i és perfectament possible aconseguir ràtios de 10 vegades, 5 vegades, o fins i tot millors (fes *make* usant el **Makefile** per a compilar).